Department Of CS

SEP 2025-26

BCA - 3RD SEMESTER

JAVA Programming Lab Manual



Mr. Guruprasad T R

Asst. Professor,
BiTS Hi-TECH College, Chitradurga
9845634942
Department Of CS

LABORATORY MANUAL

FOR

JAVA PROGRAMMING

III Semester

SEP 2025-26

Bachelor Of Computer Applications

BITS Hi-TECH COLLEGE

(Affiliated to Davangere University & Approved by AICTE)

Opposite Agriculture Office, VP Extension, Chitradurga – 577501

Prepared By : Guruprasad T R, Asst. Professor, 9845634942

BCA 3rd Semester			Java Programming Lab	
Subject Code:	BCA 2024 BC 05		Total Teaching Hours:	84
IA Marks:	20		Teaching Hours/Week:	03*2 = 6 Hrs
Exam Marks:	80		Examination Hours:	03
Credits:	3			

PART-A
1. Programs Using Different Control Structures (Switch, If, While, Do, For etc.,) 2. Programs Using Arrays. 3. Programs Using Strings, String Buffer Classes and Vectors. 4. Programs using constructor and destructor. 5. Creation of classes and use of different types of functions. 6. Count the number of objects created for a class using static member function. 7. Write programs on interfaces and abstract classes. 8. Write programs on packages and importing. 9. Write programs using function overloading and overriding. 10. Programs using inheritance.
PART-B
1. Write a Java program that prompts the user for an integer and then prints out all the prime numbers up to that Integer? 2. Write a Java program that checks whether a given string is a palindrome or not. Ex: MADAM is a palindrome? 3. Write a Java program for sorting a given list of names in ascending order? 4. Write a Java program to multiply two given matrices? 5. Write a Java program that reads a line of integers and then displays each integer and the sum of all integers. (use StringTokenizer class)? 6. Write a Java program that reads on file name from the user, then displays information about whether the file exists, whether the file is readable, whether the file is writable, the type of file and the length of the file in bytes? 7. Write a Java program that displays the number of characters, lines and words in a text? 8. JSP Program to select record from database. 9. JSP Program to display given number in words. 10. Write an application that displays deadlock between threads. Program to display data from database table using JDBC.

Examination:		
• Two Questions has to be given from the above list A and B (one each).		
• Student has to write and execute both of the programs		
Marks Distribution:		
Criteria		**Marks**
		Questions from The List
Practical Proper	Writing & Execution of Program one from PART-A & one from PART-B	60 (Each 30 marks)
	Total	**60**
Viva		20
Total		**80**

1. Programs Using Different Control Structures (Switch, If, While, Do, For etc.,)

a. Program to simulate simple calculator using while and switch case

```
import java.util.Scanner;

class Calculator {
  public static void main(String[] args) {
    char operator;
    double number1, number2;

    Scanner input = new Scanner(System.in);

    System.out.println("Enter first number");
    number1 = input.nextDouble();

    System.out.println("Choose an operator: +, -, *, / or =");
    operator = input.next().charAt(0);

    while (operator!= '=')
    {
          System.out.println("Enter second number");
          number2 = input.nextDouble();

          switch (operator) {

      case '+':
                        number1 += number2;
                        break;
      case '-':
                        number1 -= number2;
                        break;
      case '*':
                        number1 *= number2;
                        break;
      case '/':
                        number1 /= number2;
                        break;
                }
      System.out.println("Choose an operator: +, -, *, / or =");
      operator = input.next().charAt(0);
    }
     System.out.println("Result :"+number1) ;
    }
}
```

```
E:\gp\Java\loops\while>javac Calculator.java

E:\gp\Java\loops\while>java  Calculator
Enter first number
10
Choose an operator: +, -, *, / or =
+
Enter second number
8
Choose an operator: +, -, *, / or =
*
Enter second number
4
Choose an operator: +, -, *, / or =
/
Enter second number
5
Choose an operator: +, -, *, / or =
-
Enter second number
6
Choose an operator: +, -, *, / or =
=
Result :8.4
```

b. Program to prints Floyd's triangle based on user inputs

```
import java.util.Scanner;

class FloydTriangle
{
   public static void main(String args[])
   {
     int rows, number = 1, counter, j;
     Scanner input = new Scanner(System.in);
     System.out.println("Enter the number of rows for Floyd's triangle:");
     rows = input.nextInt();
     System.out.println("Floyd's triangle");
     System.out.println("****************");
     for ( counter = 1 ; counter <= rows ; counter++ )
     {
        for ( j = 1 ; j <= counter ; j++ )
        {
            System.out.print(number+" ");
            number++;
        }
        System.out.println();
     }
  }
}
```

```
E:\gp\Java\loops\for>java  FloydTriangle
Enter the number of rows for floyd's triangle:
5
Floyd's triangle
****************
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

2. Programs Using Arrays.

Write a program to find sum of two matrix of order mXn.

```
import java.util.Scanner;
public class MatrixSum
{
   public static void main(String[] args)
   {
      int m, n;
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the number of rows and columns of matrix:");
      m = sc.nextInt();   //Initialize second matrix size
      n = sc.nextInt();   //Initialize second matrix size
         int a[][] = new int[m][n];    //Declare first matrix
         int b[][] = new int[m][n];    //Declare second matrix
         int c[][] = new int[m][n];    //Declare third matrix

         System.out.println("Enter all the elements of first matrix:");
         for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
               a[i][j] = sc.nextInt();

         System.out.println("");
         System.out.println("Enter all the elements of second matrix:");
         for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
               b[i][j] = sc.nextInt();
         System.out.println("");

         System.out.println("Matrix after addition:");
         for (int i = 0; i < m; i++)
         {
            for (int j = 0; j < n; j++)
            {
               System.out.print((a[i][j] + b[i][j]) +" ");
            }
            System.out.println("");
         }
      }
}
```

```
E:\gp\Java\Arrays\Matrix>javac MatrixSum.java

E:\gp\Java\Arrays\Matrix>java  MatrixSum
Enter the number of rows and columns of matrix:3
3
Enter all the elements of first matrix:
1 4 7
2 5 8
3 6 9

Enter all the elements of second matrix:
1 1 1
2 2 2
3 3 3

Matrix after addition:
2 5 8
4 7 10
6 9 12
```

3. Programs Using Strings, String Buffer Classes and Vectors.

a. **Program to Read two String from user input and check if first contains second.**

```
import java.util.Scanner;

public class StringContains {

	public static void main(String[] args) {
		Scanner scanner = new Scanner(System.in);
		System.out.println("Enter First String:");
		String s1 = scanner.nextLine();

		System.out.println("Enter Second String:");
		String s2 = scanner.nextLine();

		boolean result = stringCheck(s1, s2);
		System.out.println(s1+" contains "+s2+" = "+result);
	}

	private static boolean stringCheck(String s1, String s2) {
		boolean result = false;
		result = s1.contains(s2);
		return result;
	}
}
```

```
E:\gp\Java\Strings>java  StringContains
Enter First String:
International
Enter Second String:
national
International contains national = true

E:\gp\Java\Strings>java  StringContains
Enter First String:
international
Enter Second String:
city
international contains city = false
```

b. Program to perform some operations on Vector

```
import java.util.Vector;

public class VectorTest1 {
public static void main(String[] args)
{
   Vector<String> v = new Vector<String>();
    v.add("A");
    v.add("B");
    v.add("C");
    v.add("D");
    v.add("E");
   System.out.println("Elements: " +v);

   String first = v.firstElement();
   System.out.println("First Element: " +first);
   String last = v.lastElement();
   System.out.println("Last Element: " +last);
   String element = v.elementAt(3);
   System.out.println("Element at position 3: " +element);
   boolean checkElement = v.contains("F");
   System.out.println("Element F: " +checkElement);

// Create an array object with an initial capacity of 5.
   String[] arr = new String[5];

// Copy collection of elements into an array str.
    v.copyInto(arr);
    System.out.println("Elements in an array arr");
    for(String str:arr)
    {
      System.out.println(str);
    }
// Call clone() method to create a clone of a vector.
   Object obj = v.clone(); // Return type of clone method is an Object.
   System.out.println("Clone of v: " +obj);
  }
}
```

```
E:\gp\Java\vector>java  VectorTest
Size of vector before adding elements: 0
Vector elements: [Red, Green, Yellow, Pink, White]
Size: 5
Default capacity: 10
New size after adding elements: 8
New capacity after adding elements: 10
Elements order after addition: [Red, Green, Yellow, Pink, White, 15, null, 20]
Size: 8
Vcapacity: 10
Elements: [Red, Green, Yellow, Pink, White, 15, null, 20, 10]
Size after adding 9th element: 9
Capacity after adding 9th element: 10
```

4. Programs using constructor and destructor.

```
class Shoes {

  private String Brand;
  private int Size;

  /* constructor */
  public Shoes(String Brand, int Size) {
     this.Brand = Brand;
     this.Size = Size;
  }

  public static void main(String[] args) {
     Shoes myshoes = new Shoes("nike", 10);
     System.out.println("Brand - "+myshoes.Brand);
     System.out.println("Size - "+myshoes.Size);

     myshoes = null;

     /* request to garbage collector */
     System.gc();
  }

  /* @override the finalize method of 'Object' class */
  protected void finalize() throws Throwable {
     System.out.println("myshoes object is destroyed");
  }
}
```

```
E:\gp\Java\ClassObject>java  Shoes
Brand - nike
Size - 10
myshoes object is destroyed
```

5. Creation of classes and use of different types of functions.

```
import java.util.Scanner;

class Box
{
   double width;
   double height;
   double length;
   void readData()
   {
      Scanner s = new Scanner(System.in);
      System.out.print("Enter length width and height : ");
       length = s.nextDouble();
       width = s.nextDouble();
       height = s.nextDouble();
   }
   void area()
   {
        System.out.println("Area  of Rectangle Box: "+(length*width));
   }
  void volume()
   {
      System.out.println("Volume  of Rectangle Box: "+(length*width*height));
   }

}
class RectangleDemo
{
   public static void main(String args[])
   {
      Box cuboid = new Box();
      cuboid.readData();
      cuboid.area();
      cuboid.volume();
   }
}
```

```
E:\gp\Java\ClassObject>java  RectangleDemo
Enter length width and height : 2.5
3.5
4.5
Area  of Rectangle Box: 8.75
Volume  of Rectangle Box: 39.375
```

6. Count the number of objects created for a class using static member function.

```
public class Counter {
   private static int count = 0;

   public Counter() {
      count++;
   }

   public static int getCount() {
      return count;
   }

   public static void main(String[] args) {
      // Create several Counter objects
      Counter c1 = new Counter();
         System.out.println("Count: " + Counter.getCount());
      Counter c2 = new Counter();
         System.out.println("Count: " + Counter.getCount());
      Counter c3 = new Counter();
      System.out.println("Count: " + Counter.getCount());
   }
}
```

```
E:\gp\Java\ClassObject>javac Counter.java

E:\gp\Java\ClassObject>java  Counter
Count: 1
Count: 2
Count: 3
```

7. Write programs on interfaces and abstract classes.

```
// Implementation of Interface using Abstract Class in Java
interface College {
    void learnDSA();
    void learnWebDevelopment();
    void share();
}
abstract class Student implements College {

     public void learnDSA() {
        System.out.println("You must have a good command on DSA to crack good
companies");
    }
     public void learnWebDevelopment() {
        System.out.println("Coding provides excellent content to master Web Development");
    }
}

class Friends extends Student {
    public void share() {
        System.out.println("Helping others to learn will make you learn too!");
    }
}
// Driver code
public class InterfaceAbstractDemo {
    public static void main(String[] args) {
        Friends student1 = new Friends();
        student1.learnDSA();
        student1.learnWebDevelopment();
        student1.share();
    }
}
```

```
E:\gp\Java\Interface>java  InterfaceAbstractDemo
You must have a good command on DSA to crack good companies
Coding provides excellent content to master Web Development
Helping others to learn will make you learn too!
```

8. Write programs on packages and importing.
Create a package 'BCA' in your current working directory

a. Create a default class student in the above package with the following attributes: Name, age, sex.
b. Have methods for storing as well as displaying

a.

```
package BCA;
public class Student
{
        int age;
        char gender;
        String name;
        public Student(String a, int e, char f)
        {
                name=a;
                age=e;
        gender=f;
        }
        public void display()
        {
                System.out.println("STUDENT PERSONAL DETAILS");
                System.out.println("Name of the student is: "+name);
                System.out.println("Age="+age);
                System.out.println("Student is "+gender);
        }
}
```

File name - Student.java

b.

```
import BCA.*;
class PackDemo
{
        public static void main(String args[])
        {
                BCA.Student a= new BCA.Student("AMITH",21,'M');
                a.display();
        }
}
```

```
E:\gp\Java>java  PackDemo
STUDENT PERSONAL DETAILS
Name of the student is: Rita
Age=18
Student is F
```

9. Write programs using function overloading and overriding.

a. Program to add two integers, two float numbers and three integers to calculate the sum. Use function overloading.

```
public class Overload
{
        public int sum(int x, int y)
        {
                return (x + y);
        }
        public int sum(int x, int y, int z)
        {
                return (x + y + z);
        }
        public double sum(double x, double y)
        {
                return (x + y);
        }
        public static void main(String args[])
        {
                Overload s = new Overload();
                System.out.println("Sum of two integers = "+ s.sum(10, 20));
                System.out.println("Sum of three integers = "+ s.sum(10, 20, 30));
        System.out.println("Sum of two floating point numbers = "+ s.sum(10.5, 20.5));
        }
}
```

```
E:\gp\Java\ClassObject>javac Overload.java

E:\gp\Java\ClassObject>java  Overload
Sum of two integers = 30
Sum of three integers = 60
Sum of two floating point numbers = 31.0
```

b. Implementation in Java using method overriding

```
class Human
{
        public void job()
        {
                System.out.println("Working as a Professional");
        }
}
class Teacher extends Human
{
        public void job()
        {
                System.out.println(" Working as a Teacher");
        }
}
class Doctor extends Human
{
        public void job()
        {
                System.out.println("Working as a Doctor");
        }
}
public class MethodOverride
{
        public static void main(String[] args)
        {
                Human h1 = new Human();
                Teacher t1 = new Teacher();
                Doctor d1 = new Doctor();
                Human h2 = new Doctor();
                h1.job();       // method invoked based on human object
                t1.job();         // method invoked based on teacher object
                d1.job();         // method invoked based on doctor object
                h2.job();         // method invoked based on doctor object
         }
}
```

```
E:\gp\Java\INHERITANCE>javac MethodOverride.java

E:\gp\Java\INHERITANCE>java  MethodOverride
Working as a Professional
 Working as a Teacher
Working as a Doctor
Working as a Doctor
```

10. Programs using inheritance.

Program to perform mathematical operations. Create a class called AddSub with methods to add and subtract. Create another class called MulDiv that extends from AddSub class to use the member data of the super class. MulDiv should have methods to multiply and divide A main function should access the methods and perform the mathematical operations.

```
class AddSub
{
	int num1;
	int num2;
	AddSub(int n1, int n2)
	{
		num1 = n1;
		num2 = n2;
	}
	void addition()
	{
		System.out.println("Sum :" + (num1+num2) );
	}
	 void subtraction()
	{
		System.out.println("Difference :" + (num1-num2) );
	}
}

class MultDiv extends AddSub
{
	public MultDiv(int n1, int n2)
	{
	super(n1, n2);
	}
	void multiplication()
	{
		System.out.println("Product :" + (num1*num2) );
	}
	void division()
	{
		System.out.println("Quotient :" + (float) (num1) /num2 );
	}
```

```
        public void display()
        {
                System.out.println("Number 1 :" + num1);
                System.out.println("Number 2 :" + num2);
        }
}

public class SIDemo
{
        public static void main(String arg[])
        {
        MultDiv r1=new MultDiv (50,20);
        r1.display();
        r1.addition();
        r1.subtraction();
        r1.multiplication();
        r1.division();
        }
}
```

```
E:\gp\Java\INHERITANCE>javac SIDemo1.java

E:\gp\Java\INHERITANCE>java  SIDemo1
Number 1 :50
Number 2 :20
Sum :70
Difference :30
Product :1000
Quotient :2.5
```

PART – B

1. Write a Java program that prompts the user for an integer and then prints out all the prime numbers up to that Integer?

```
import java.util.Scanner;

class PrimeNumbers
{
	public static void main(String[] args)
	{
		int n;
		int p;
		Scanner s=new Scanner(System.in);
		System.out.println("Enter a number: ");
		n=s.nextInt();
		System.out.println("Prime numbers ");
		for(int i=1;i<n;i++)
		{
			p=0;
			for(int j=2;j<i;j++)
			{
				if(i%j==0)
				p=1;
			}
			if(p==0)
				System.out.println(i);
		}
	}
}
```

```
E:\gp\Java\loops\for>javac PrimeNumbers.java

E:\gp\Java\loops\for>java  PrimeNumbers
Enter a number:
50
Prime numbers
1
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
```

2. Write a Java program that checks whether a given string is a palindrome or not. Ex: MADAM is a palindrome?

```
import java.util.Scanner;

public class Palindrome {
        public static void main(String[] args) {
                Scanner in = new Scanner(System.in);
                System.out.println("Enter a string to check: ");
                String s = new String(in.next());
                int i,j;
                j=s.length()-1;
                for (i =0 ; j >=i ; j--,i++)
                {
                        if(s.charAt(i)!=s.charAt(j))
                        {
                                System.out.println(s+" is not palindrome");
                                System.exit(1);
                        }
                }

                System.out.println(s+" String is palindrome");
        }
}
```

```
E:\gp\Java\Strings>javac Palindrome.java

E:\gp\Java\Strings>java  Palindrome
Enter a string to check:
MADAM
MADAM String is palindrome

E:\gp\Java\Strings>java  Palindrome
Enter a string to check:
BITS
BITS is not palindrome
```

3. Write a Java program for sorting a given list of names in ascending order?

```
import java.util.*;
class Sorting
{
        int n=0;
        String []str;
        void readStrings()
        {
                Scanner s = new Scanner(System.in);
                System.out.println("Enter the value of n: ");
                n = s.nextInt();
                str = new String[n];
                System.out.println("Enter strings: ");
                for(int i = 0; i < n; i++)
                        str[i] = new String(s.next());
        }
        void sortStrings()
        {
                for(int i = 0; i < n; i++)
                {
                        for(int j = i+1; j < n; j++)
                        {
                                if(str[i].compareTo(str[j])>0)
                                {
                                        String temp = str[i];
                                        str[i] = str[j];
                                        str[j] = temp;
                                }
                        }
                }
        }
        void showStrings()
        {
                for(int i = 0; i < n ; i++)
                        System.out.println(str[i]);
        }
}
class StringSort
{
```

```
        public static void main(String[] args)
        {
                Sorting obj = new Sorting();
                obj.readStrings();
                System.out.println("Before Sort list of strings :");
                obj.showStrings();
                obj.sortStrings();
                System.out.println("After Sort list of strings :");
                obj.showStrings();
        }
}
```

```
E:\gp\Java\Strings>javac StringSort.java

E:\gp\Java\Strings>java  StringSort
Enter the value of n:
5
Enter strings:
Sunil
Ram
Amith
Ravi
Abhinva
Before Sort list of strings :
Sunil
Ram
Amith
Ravi
Abhinva
After Sort list of strings :
Abhinva
Amith
Ram
Ravi
Sunil
```

4. Write a Java program to multiply two given matrices.

```
/*Java Program to multiply two matrices*/
import java.util.Scanner;
class Matrix
{
	int  p, q, m, n;
	int a[ ][ ], b [ ][ ], c[ ][ ];
	int i,j,k;
	void readMatrix()
	{
		Scanner sc = new Scanner(System.in);
		System.out.print("Enter the number of rows in the first matrix:");
		p= sc.nextInt();
		System.out.print("Enter number of columns in the first matrix:");
		q = sc.nextInt();
		System.out.print("Enter the number of rows in the second matrix:");
		m = sc.nextInt();
		System.out.print("Enter the number of columns in the second matrix:");
		n = sc.nextInt();
		a = new int[p][q];
		b = new int[m][n];
		System.out.println("Enter all the elements of first matrix:");
		for (int i = 0; i < p; i++)
			for (int j = 0; j < q; j++)
				a[i][j] = sc.nextInt();

		System.out.println("Enter all the elements of second matrix:");
		for (int i = 0; i < m; i++)
			for (int j = 0; j < n; j++)
				b[i][j] = sc.nextInt();
	}
	void showMatrix()
	{
		System.out.println("First Matrix:");
		for(int i=0;i<p;i++)
		{
			for(int j=0;j<q;j++)
				System.out.print(a[i][j]+" ");
			System.out.println("");
		}
```

```
                System.out.println("Second Matrix:");
                for(int i=0;i<m;i++)
                {
                        for(int j=0;j<n;j++)
                                System.out.print(b[i][j]+" ");
                        System.out.println("");
                }
        }
        void productMatrix()
        {
                if (m != q) {
                        System.out.println("Multiplication Not Possible");
                        return;
        }
        int c[][] = new int[q][n];
        k=0;
        for(int i=0;i<p;i++)
        {
                for(int j=0;j<n;j++)
                {
                        for (k = 0; k < m; k++)
                                c[i][j] += a[i][k] * b[k][j];
                }
        }
        System.out.println("Resultant Matrix:");
        for(int i=0;i<p;i++)
        {       for(int j=0;j<n;j++)
                        System.out.print(c[i][j]+" ");
                System.out.println();
        }
}
}
public class MatrixMultiplication  {
        public static void main(String[] args) {
                Matrix obj = new Matrix();
                obj.readMatrix();
                obj.showMatrix();
                obj.productMatrix();
        }
}
```

```
E:\gp\Java\Arrays\Matrix>java  MatrixMultiplication
Enter the number of rows in the first matrix:2
Enter number of columns in the first matrix:2
Enter the number of rows in the second matrix:2
Enter the number of columns in the second matrix:2
Enter all the elements of first matrix:
1 4
2 5
Enter all the elements of second matrix:
3 2
2 3
First Matrix:
1 4
2 5
Second Matrix:
3 2
2 3
Resultant Matrix:
11 14
16 19
```

```
E:\gp\Java\Arrays\Matrix>java  MatrixMultiplication
Enter the number of rows in the first matrix:2
Enter number of columns in the first matrix:3
Enter the number of rows in the second matrix:2
Enter the number of columns in the second matrix:3
Enter all the elements of first matrix:
1 1 1
2 2 2
Enter all the elements of second matrix:
3 3 3
1 1 1
First Matrix:
1 1 1
2 2 2
Second Matrix:
3 3 3
1 1 1
Multiplication Not Possible
```

5. Write a Java program that reads a line of integers and then displays each integer and the sum of all integers. (use StringTokenizer class)?

```
import java.util.Scanner;

public class IntegerSum
{
        public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        System.out.println("Enter integers separated by spaces:");

        String line = s.nextLine();

        String[] tokens = line.split(" ");

        int sum = 0;

        System.out.println("Given Integers entered :");

        for (String token : tokens)
        {
                int num = Integer.parseInt(token);
                System.out.print(num + " ");
                sum += num;
        }

        System.out.println("\nSum = " + sum);
  }
}
```

```
E:\gp\Java\Strings>javac IntegerSum.java

E:\gp\Java\Strings>java  IntegerSum
Enter integers separated by spaces:
10 5 -3 8 9 -5
Given Integers entered :
10 5 -3 8 9 -5
Sum = 24
```

6. Write a Java program that reads on file name from the user, then displays information about whether the file exists, whether the file is readable, whether the file is writable, the type of file and the length of the file in bytes?

```
import java.io.File;
import java.util.Scanner;

public class FileInfo
{
	public static void main(String[] args) {
	Scanner scanner = new Scanner(System.in);
	System.out.print("Enter the file name (with path if necessary): ");
	String fileName = scanner.nextLine();

	File file = new File(fileName);

	if (file.exists())
	{
		System.out.println("File exists: Yes");
		System.out.println("Readable: " + (file.canRead() ? "Yes" : "No"));
		System.out.println("Writable: " + (file.canWrite() ? "Yes" : "No"));

		if (file.isDirectory())
			System.out.println("Type: Directory");
		else
			System.out.println("Type: File");

		System.out.println("Length: " + file.length() + " bytes");
	}
	else
	{
		System.out.println("File exists: No");
	}
    }
}
```

```
E:\gp\Java\file>java  FileInfo
Enter the file name (with path if necessary): FileInfo.java
File exists: Yes
Readable: Yes
Writable: Yes
Type: File
Length: 949 bytes

E:\gp\Java\file>java  FileInfo
Enter the file name (with path if necessary): StudentInfo
File exists: No
```

7. Write a Java program that displays the number of characters, lines and words in a text file

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;

public class TextAnalyzer
{
	public static void main(String[] args)
	{
		Scanner s = new Scanner(System.in);
		System.out.print("Enter the file name (with path if necessary): ");
		String filePath = s.nextLine();
		int charCount = 0;
		int wordCount = 0;
		int lineCount = 0;
		try
		{
			BufferedReader reader = new BufferedReader(new FileReader(filePath));
			String line;
			while ((line = reader.readLine()) != null) {
				lineCount++;
				charCount += line.length();
				String[] words = line.split("\\s+");

				for (String word : words) {
					if (!word.isEmpty()) {
						wordCount++;
					}
				}
			}
		}
		 catch (IOException e)
		{
			System.err.println("Error reading the file: " + e.getMessage());
		}

		System.out.println("Analysis of " + filePath + ":");
		System.out.println("Number of Characters: " + charCount);
```

```
            System.out.println("Number of Words: " + wordCount);
            System.out.println("Number of Lines: " + lineCount);
        }
}
```

```
E:\gp\Java\file>javac TextAnalyzer.java

E:\gp\Java\file>java  TextAnalyzer
Enter the file name (with path if necessary): TextAnalyzer.java
Analysis of TextAnalyzer.java:
Number of Characters: 1750
Number of Words: 207
Number of Lines: 46

E:\gp\Java\file>java  TextAnalyzer
Enter the file name (with path if necessary): demo.txt
Error reading the file: demo.txt (The system cannot find the file specified)
Analysis of demo.txt:
Number of Characters: 0
Number of Words: 0
Number of Lines: 0
```

8. Write a JSP application to select the record from the database.

Answer:

Below example shows how to connect JSP program to oracle database.

There is an Employee table in database, we fetch the all record from the database table and print them.

1. Create the Employee table

Create table Employees (id number, age number, FirstName varchar2(20), LastName varchar2(20));

2. Create the JSP application

JspDB.jsp

```
<%@ page import="java.sql.*"%>
<%@ page import="java.util.*"%>
<html>
   <head>
      <title>SELECT Operation</title>
   </head>
   <body>
      <%!
         Connection con;
         PreparedStatement ps;
         public void jspInit()
         {
            try
            {
               Class.forName("oracle.jdbc.driver.OracleDriver");
               con =
DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:XE", "local", "test");
               ps = con.prepareStatement("select * from Employees");
            }
            catch(Exception ex)
            {
               ex.printStackTrace();
            }
         }
      %>
```

```
<%
    ResultSet rs = ps.executeQuery();
    out.println("<table border=1 >");
    out.println("<tr style='background-color:#ffffb3; color:red'>");
    out.println("<th>Emp Id</th>");
    out.println("<th> Age(Year)</th>");
    out.println("<th>First Name</th>");
    out.println("<th>Last Name</th>");
    out.println("</tr>");

    while(rs.next())
    {
      out.println("<tr style='background-color:#b3ffd9;'>");
      out.println("<td>"+rs.getInt(1)+"</td>");
      out.println("<td>"+rs.getInt(2)+"</td>");
      out.println("<td>"+rs.getString(3)+"</td>");
      out.println("<td>"+rs.getString(4)+"</td");
      out.println("</tr>");
    }
    out.println("</table>");
    rs.close();
  %>
 </body>
</html>
```

web.xml

```
<web-app>
  <servlet>
    <servlet-name>xyz</servlet-name>
    <jsp-file>/JspDB.jsp</jsp-file>
  </servlet>
  <servlet-mapping>
    <servlet-name>xyz</servlet-name>
    <url-pattern>/test</url-pattern>
  </servlet-mapping>
</web-app>
```

Output:

Emp Id	Age(Year)	First Name	Last Name
100	22	Amit	Kumar
101	25	Anuj	Gupta
102	26	Viraj	Sharma
103	23	Jitu	Singh
104	32	Adesh	Pandey

9. Create a JSP page to accept a number from the user and display it in words.
Example: 123 - One Two Three.

Answer:

The index.html taking the number from the user and then change it into the words.

index.html

```
<!DOCTYPE html>
<html>
    <body>
        <form method=get action="test.jsp">
            Enter Any Number : <input type=text name=num><br><br>
            <input type=submit value="Display">
        </form>
    </body>
</html>
```

Test.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
   <body>
     <font color=red>
       <%! int i,n;
           String s1;
       %>
       <%   s1= request.getParameter("num");
           n=s1.length();
           i=0;
           do
           {
               char ch=s1.charAt(i);
               switch(ch)
               {
                   case '0': out.println("Zero  ");
                       break;
                   case '1': out.println("One  ");
                       break;
                   case '2': out.println("Two  ");
                       break;
```

```
                        case '3': out.println("Three  ");
                            break;
                        case '4': out.println("Four ");
                            break;
                        case '5': out.println("Five  ");
                            break;
                        case '6': out.println("Six  ");
                            break;
                        case '7': out.println("Seven  ");
                            break;
                        case '8': out.println("Eight  ");
                            break;
                        case '9': out.println("Nine  ");
                            break;
                    }
                    i++;
                }
                while(i<n);
            %>
        </font>
    </body>
</html>
```

web.xml

```
<web-app>
    <servlet>
        <servlet-name>xyz</servlet-name>
        <jsp-file>/test.jsp</jsp-file>
    </servlet>
    <servlet-mapping>
        <servlet-name>xyz</servlet-name>
        <url-pattern>/test</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

Output:



Nine Five Three Two One Zero

10. Write an application that displays deadlock between threads. Program to display data from database table using JDBC.

Application Displaying Deadlock Between Threads and Data from Database using JDBC

This application demonstrates a classic deadlock scenario between two threads and also includes functionality to display data from a database table using JDBC.

1. Deadlock Demonstration:
This part of the application will create a deadlock by having two threads attempt to acquire locks on two shared resources in opposite orders.

```
import java.lang.*;

public class DeadlockDemo {

   private static final Object resource1 = new Object();
   private static final Object resource2 = new Object();

   public static void main(String[] args)
         {
         Thread thread1 = new Thread() ;
                  synchronized (resource1) {
            System.out.println("Thread 1: Acquired lock on Resource 1");
            try {
               Thread.sleep(100); // Simulate some work
            } catch (InterruptedException e) {
               e.printStackTrace();

            System.out.println("Thread 1: Waiting for lock on Resource 2");
            synchronized (resource2) {
               System.out.println("Thread 1: Acquired lock on Resource 2");
            }
         }
      };

      Thread thread2 = new Thread();
         synchronized (resource2) {
         System.out.println("Thread 2: Acquired lock on Resource 2");
            try {
               Thread.sleep(100); // Simulate some work
```

```
            } catch (InterruptedException e) {
                e.printStackTrace();

            System.out.println("Thread 2: Waiting for lock on Resource 1");
            synchronized (resource1){
                System.out.println("Thread 2: Acquired lock on Resource 1");
            }
        }
    };

    thread1.start();
    thread2.start();
  }
}
```

```
E:\gp\Java\multithreading>javac DeadlockDemo.java

E:\gp\Java\multithreading>java  DeadlockDemo
Thread 1: Acquired lock on Resource 1
Thread 2: Acquired lock on Resource 2
```

2. Displaying Data from Database Table using JDBC:

This part demonstrates how to connect to a database and retrieve data using JDBC.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCDisplayData {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/EmployeeDb";
    private static final String USER = "System";
    private static final String PASS = "Admin123";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;
```

```
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            System.out.println("Connecting to database...");
            conn = DriverManager.getConnection(DB_URL, USER, PASS);

            System.out.println("Creating statement...");
            stmt = conn.createStatement();
            String sql = "SELECT id, name, email FROM Employee";
            rs = stmt.executeQuery(sql);
            while (rs.next()) {
               int id = rs.getInt("id");
               String name = rs.getString("name");
               String email = rs.getString("email");
               System.out.println("ID: " + id + ", Name: " + name + ", Email: " + email);
            }
        }
            catch (SQLException se) {  se.printStackTrace();     }
            catch (Exception e) { e.printStackTrace();            }
            finally {
                    try {
                            if (rs != null) rs.close();
                    } catch (SQLException se2) {
                    }
                    try {
                            if (stmt != null) stmt.close();
                            } catch (SQLException se2) {
                     }
                    try {
                            if (conn != null) conn.close();
                    }
                     catch (SQLException se) {    se.printStackTrace();     }
             }
      }
}
```